| |
|---|
| Experiment No.6 |
| Develop Structure based  social media analytics model for any business. |
| Date of Performance:13/02/2025 |
| Date of Submission:20/02/2025 |

**Aim:** Develop Structure based social media analytics model for any business.

**Objective:** Develop a robust social media analytics model leveraging NetworkX in Python to analyze and visualize the structure of social networks for businesses, enabling insights into relationships, influences, and potential marketing strategies based on graph-based analysis of user interactions and connections.

**Theory:**

NetworkX is a Python package for complex graph network analysis. In order to understand NetworkX functionality, we first need to understand graphs.

What are Graphs?
Graphs are mathematical structures used to model many types of relationships and processes in physical, biological, social and information systems. A graph consists of nodes or vertices (representing the entities in the system) that are connected by edges (representing relationships between those entities). Working with graphs is a function of navigating edges and nodes to discover and understand complex relationships and/or optimize paths between linked data in a network.
There are many uses of graph network analysis, such as analyzing relationships in social networks, cyber threat detection, and identifying the people most likely to buy a product based on shared preferences.
In the real world, nodes can be people, groups, places, or things such as customers, products, members, cities, stores, airports, ports, bank accounts, devices, mobile phones, molecules, or web pages.
Examples of edges, or relationships between nodes, include friendships, network connections, hyperlinks, roads, routes, wires, phone calls, emails, "likes," payments, transactions, phone calls, and social networking messages. Edges can have a one-way direction arrow to represent a relationship from one node to another, like if Janet "liked" a social media post of Jeanette's. But they can also be non-directional, like if Bob is a Facebook friend of Alice, then Alice is also a friend of Bob.

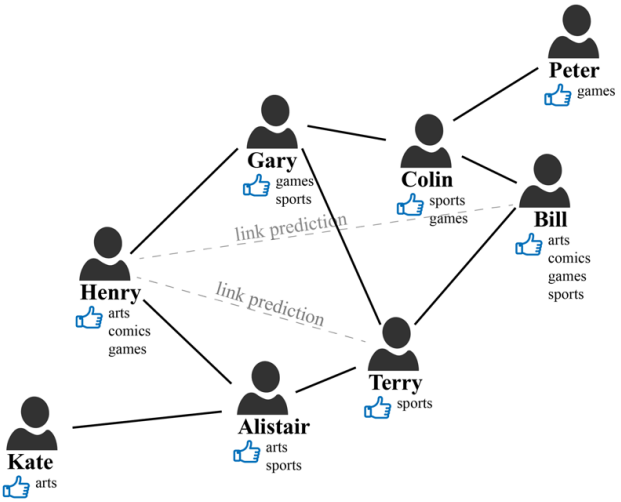



*fig. 1: Graph based example*

NetworkX nodes can be any object that is hashable, meaning that its value never changes. These can be text strings, images, XML objects, entire graphs, and customized nodes. The base package includes many functions to generate, read, and write graphs in multiple formats. NetworkX has the capacity to operate on very large graphs with more than 10 million nodes and 100 million edges. The core package, which is free software under the BSD license, includes data structures for representing such things as simple graphs, directed graphs, and graphs with parallel edges and self-loops. NetworkX also has a large community of developers who maintain the core package and contribute to a third-party ecosystem.
Among the principal uses of NetworkX are:

- Study of the structure and dynamics of social, biological, and infrastructure networks
- Standardized programming environment for graphs
- Rapid development of collaborative, multidisciplinary projects
- Integration with algorithms and code written in C, C++, and FORTRAN
- Working with large nonstandard data sets

NetworkX is considered relatively easy to install and use, particularly for Python developers.
Some functions in Networkx are:-

1) Graph.add_edge :- Used to add an edge between two nodes in a graph, allowing the creation or modification of the graph structure.
2) Graph.add_node :- Used to add a node to a graph, allowing users to specify node attributes if needed.

3) Graph.edges :- Returns a list of edges in the graph, where each edge is represented as a tuple of nodes.

4) networkx.draw(G, with_labels=1) :- Used to visualize the graph G with node labels displayed.

5) networkx.degree_centrality(G) :- Calculates the degree centrality for each node in the graph G, which is the fraction of nodes a node is connected to.

6) networkx.closeness_centrality(G) :- calculates the closeness centrality for each node in a graph, measuring the reciprocal of the average shortest path distance from a node to all other nodes in the graph.

7) networkx.degree(G) :- Returns a dictionary where keys are nodes in the graph G and values are their respective degrees, representing the number of edges incident to each node.

8) m_influential=networkx.degree_centrality(G)
networkx.eigenvector_centrality(G)
:- m_influential is a variable storing the degree centrality of nodes in graph G calculated using networkx.degree_centrality(). The function networkx.eigenvector_centrality(G) computes the eigenvector centrality of nodes in the graph, measuring their influence based on the principle that connections to high-scoring nodes contribute more to a node's score.

9) networkx.betweenness_centrality(G) :- It computes the betweenness centrality for each node in the graph G, measuring the fraction of shortest paths that pass through a node, identifying nodes that act as critical bridges in the network.

10) list(networkx.find_cliques(G)) :- Returns all cliques (complete subgraphs) in the graph G.

11) networkx.ego_graph :- It constructs the ego graph of a specified node, which includes the node and its neighbors within a given radius or distance.

12) networkx.spring_layout :- It implements a force-directed layout algorithm, positioning nodes in a graph based on attractive and repulsive forces between them to create visually appealing network visualizations.

13) networkx.density(G) :- It calculates the density of a graph G, representing the proportion of actual edges to the total number of possible edges in the graph.

14) networkx.has_bridges(G) :- It determines whether the graph G contains any bridges (edges whose removal increases the number of connected components) and returns a boolean value accordingly.

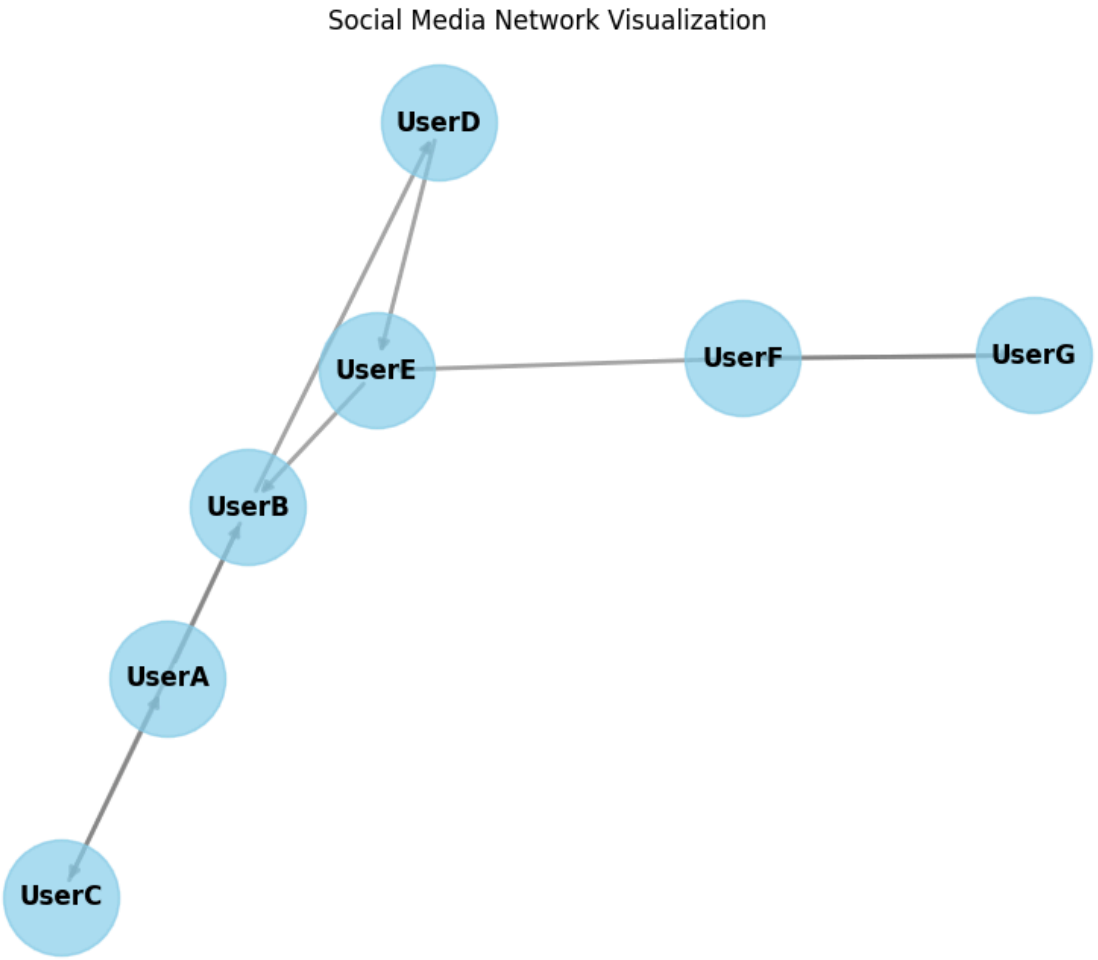
**Code and Output:**

```
import networkx as nx
import matplotlib.pyplot as plt
import pandas as pd
from networkx.algorithms.community import greedy_modularity_communities

# Example user interaction data (for simplicity, represented as a list
of edges)
# Example format: (follower, followee) or (user, user_interaction)
```

```
edges = [
    ('UserA', 'UserB'),
    ('UserB', 'UserC'),
    ('UserC', 'UserA'),
    ('UserB', 'UserD'),
    ('UserD', 'UserE'),
    ('UserE', 'UserB'),
    ('UserF', 'UserE'),
    ('UserG', 'UserF'),
    ('UserF', 'UserG')
]

# 1. Create a directed graph based on interactions
G = nx.DiGraph()
G.add_edges_from(edges)

# 2. Analyze network centrality measures (e.g., Degree Centrality,
Betweenness Centrality)
degree_centrality = nx.degree_centrality(G)
betweenness_centrality = nx.betweenness_centrality(G)

# Print centrality scores for users (influencers)
print("Degree Centrality:", degree_centrality)
print("Betweenness Centrality:", betweenness_centrality)

# 3. Community detection (e.g., using modularity)
communities = list(greedy_modularity_communities(G))
print("\nDetected Communities:")
for i, community in enumerate(communities):
    print(f"Community {i + 1}: {community}")

# 4. Visualize the network
plt.figure(figsize=(10, 8))
pos = nx.spring_layout(G)  # Layout for visualization

# Draw nodes, edges, and labels
nx.draw_networkx_nodes(G, pos, node_size=3000, node_color='skyblue',
alpha=0.7)
nx.draw_networkx_edges(G, pos, width=2, alpha=0.7, edge_color='gray')
```

```
nx.draw_networkx_labels(G, pos, font_size=12, font_weight='bold',
font_color='black')

# Display the plot
plt.title("Social Media Network Visualization")
plt.axis('off')  # Turn off axis
plt.show()

# 5. Shortest Path Example: How to measure influence spread
# Find the shortest path from UserA to UserE (if any)
shortest_path = nx.shortest_path(G, source='UserA', target='UserE',
method='dijkstra')
print("\nShortest Path from UserA to UserE:", shortest_path)

# 6. Insights for Marketing Strategy:
# - Influencers: Users with high degree centrality or betweenness
centrality
# - Targeted campaigns can be aimed at users in the same community.
```

```
Degree Centrality: {'UserA': 0.3333333333333333, 'UserB':
0.6666666666666666, 'UserC': 0.3333333333333333, 'UserD':
0.3333333333333333, 'UserE': 0.5, 'UserF': 0.5, 'UserG':
0.3333333333333333}
Betweenness Centrality: {'UserA': 0.1, 'UserB': 0.5333333333333333,
'UserC': 0.16666666666666666, 'UserD': 0.1, 'UserE':
0.36666666666666664, 'UserF': 0.16666666666666666, 'UserG': 0.0}

Detected Communities:
Community 1: frozenset({'UserC', 'UserB', 'UserA'})
Community 2: frozenset({'UserD', 'UserE'})
Community 3: frozenset({'UserG', 'UserF'})
```




```
Shortest Path from UserA to UserE: ['UserA', 'UserB', 'UserD', 'UserE']
```

**Conclusion:**

The development of a robust social media analytics model using NetworkX in Python proves to be a valuable tool for businesses looking to understand and optimize their social media strategies. By leveraging graph-based analysis, the model effectively maps and visualizes the structure of social networks, revealing intricate details about user interactions, relationships, and influences within the community. This insight empowers businesses to identify key influencers, target specific audience segments, and craft data-driven marketing strategies. The model's ability to process large datasets and dynamically update as the social network evolves makes it a powerful asset for ongoing analysis and decision-making, offering businesses a competitive edge in their marketing efforts and customer engagement initiatives.